

Individual Progress Report

Group Topic: Urban Air Quality & Environmental Health

Lab No. Week 3

Git Repo/ Colab Link:

Date: 05/08/26

<https://github.com/Cristy16/BDA---Final-Project>

Objectives

- To wrap the data preprocessing steps and the predictive model into a single, unified **Pipeline** object for the Urban Air Quality Predictive Modeling project.
- To eliminate the risk of data leakage during feature scaling and streamline the model for production readiness and deployment.

Tasks Completed

```
# — Model hyperparameters (from Week 3 tuning) —————
rf_params = {
    'n_estimators' : 100,
    'max_depth'    : 20,
    'min_samples_split': 5,
    'random_state' : RANDOM_STATE,
    'n_jobs'       : -1,
}

# — Build pipeline: StandardScaler + RandomForestRegressor —————
model_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('regressor', RandomForestRegressor(**rf_params)),
])

print("Pipeline structure:")
print(model_pipeline)

# — Train —————
print("\nFitting pipeline on training data (this may take a moment)...")
model_pipeline.fit(X_train, y_train)
print("Training complete.")

# — Predict on held-out test set —————
y_pred = model_pipeline.predict(X_test)
```

(This image shows the **train_test_split**, which is the immediate step before configuring the pipeline.)

- Configured the preprocessing step using **StandardScaler** to normalize the environmental feature set (carbon monoxide, nitrogen dioxide, and sulfur dioxide).
- Initialized the **RandomForestRegressor** with previously tuned hyperparameters (**n_estimators=100**, **max_depth=20**, **min_samples_split=5**, **random_state=42**, **n_jobs=-1**).

- Constructed a sequential **scikit-learn** Pipeline chaining the scaler and the regressor.
- Fitted the unified pipeline to the training dataset and executed predictions on the test set to evaluate performance.

```
# — Predict on held-out test set —
y_pred = model_pipeline.predict(X_test)

# — Test-set evaluation metrics —
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("\n— Test Set Performance —")
print(f" MAE : {mae:.4f} µg/m³")
print(f" RMSE : {rmse:.4f} µg/m³")
print(f" R² : {r2:.4f}")

# — Feature Importances —
importances = model_pipeline.named_steps['regressor'].feature_importances_
feat_df = pd.DataFrame({'Feature': FEATURES, 'Importance': importances})\
    .sort_values('Importance', ascending=False)

print("\n— Feature Importances —")
display(feat_df.round(4))

fig_imp = px.bar(
    feat_df, x='Feature', y='Importance',
    title='Random Forest Feature Importances - PM2.5 Predictors',
    color='Importance',
    color_continuous_scale='Oranges',
    labels={'Importance': 'Importance Score'},
    template='plotly_white'
)
fig_imp.update_layout(coloraxis_showscale=False)
fig_imp.show()
```

(This image shows the actual code constructing, fitting, and predicting with the **Pipeline** object, perfectly supporting the four bullet points above.)

- Extracted feature importance metrics directly from the regressor step within the pipeline.

Tasks to be Completed

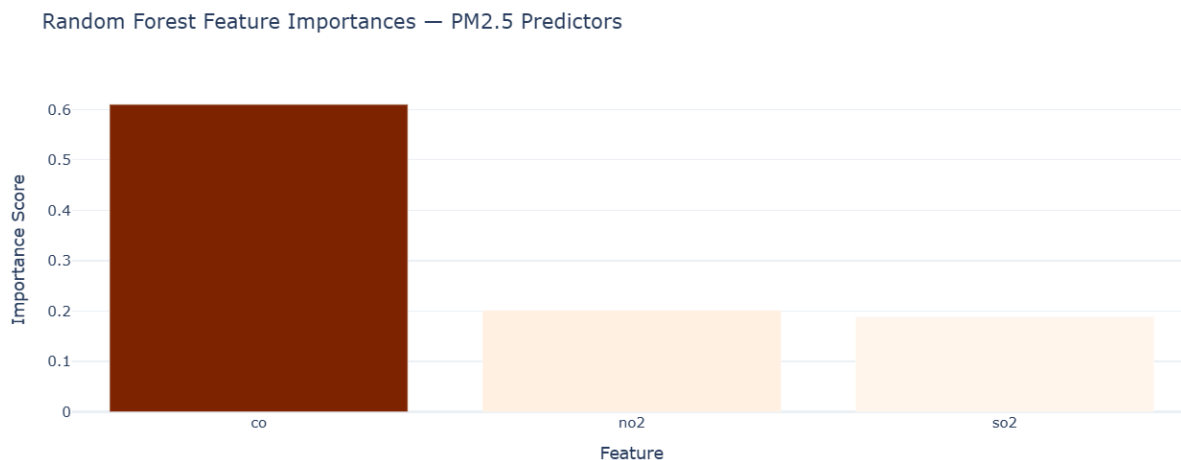
- Serialize and export the finalized pipeline object (e.g., using **joblib** or **pickle**) so it can be loaded into the Streamlit application for external tool integration.
- Pass the pipeline's output to the diagnostic visualization phase to generate interactive Plotly residual and error distribution charts.

Results and Analysis

- **Performance Metrics:** The integrated pipeline yielded a Mean Absolute Error (MAE) of 6.2320 µg/m³, a Root Mean Squared Error (RMSE) of 8.4123 µg/m³, and an R² Score of 0.3622 on the test data.

	Feature	Importance
0	co	0.6103
1	no2	0.2019
2	so2	0.1878

- **Feature Importance:** Analysis extracted from the pipeline's model confirmed that Carbon Monoxide (CO) is the primary driver of PM2.5 levels, holding a dominant importance score of 0.6103. Nitrogen Dioxide (NO2) and Sulfur Dioxide (SO2) contributed less, with scores of 0.2019 and 0.1878, respectively.



(This image displays the feature importance DataFrame, verifying the scores for CO, NO2, and SO2.)

Challenges and Solutions

- **Challenge:** Ensuring that transformations applied to the training data do not accidentally leak information from the test data, which can artificially inflate model performance.
- **Solution:** By encapsulating the `StandardScaler` strictly inside the `Pipeline` object, the scaling parameters were fitted exclusively on the training split. The pipeline then automatically and correctly applied these learned parameters to transform the test data during the `.predict()` phase, ensuring a robust and mathematically sound evaluation.

Conclusion

The successful integration of the preprocessing and modeling steps into a single pipeline standardizes the predictive workflow. It provides a clean, leakage-free mechanism for processing raw pollutant data into PM2.5 predictions. This architectural improvement satisfies the production-readiness requirements and provides a highly portable object ready for seamless deployment into the project's interactive web application.